

{ key : value }

Diccionarios en Python — Ejercicios Prácticos

🔑 [Crea, accede, itera y domina los diccionarios](#)

De PSeInt a Python — Guía Práctica Paso a Paso

Docente: Jhonathan Arley Peñaloza Sierra

Skill: Introducción a Python

[Crear](#) · [Acceder](#) · [Iterar](#) · [Métodos](#) · [Anidados](#)

Conceptos Clave: Diccionarios en Python

Antes de comenzar los ejercicios, repásalos cada vez que tengas dudas. Un diccionario es como un diccionario real: buscas por una palabra (key) y obtienes su significado (value).

Anatomía de un diccionario en Python:

```
# CREAR un diccionario
usuario = {
    "nombre": "Sara",    # key: "nombre" → value: "Sara"
    "edad": 27,         # key: "edad" → value: 27
    "ciudad": "Cúcuta"  # key: "ciudad" → value: "Cúcuta"
}

# ACCEDER a un valor
print(usuario["nombre"])    # → Sara
print(usuario.get("edad"))  # → 27

# MODIFICAR o AGREGAR
usuario["edad"] = 28        # cambia el valor
usuario["correo"] = "s@g.com" # agrega nueva key

# ITERAR con .items()
for k, v in usuario.items():
    print(k, "→", v)
```

Concepto	Descripción
Key (llave)	El nombre que usas para buscar. Siempre es único dentro del diccionario.
Value (valor)	El dato asociado a la key. Puede ser cualquier tipo: texto, número, lista, otro diccionario.
{ }	Llaves { } para crear el diccionario. Cada par se escribe como key: value separado por comas.
d[key]	Accede al valor directamente. Si la key no existe, genera un error (KeyError).
.get(key)	Accede al valor de forma segura. Si la key no existe, devuelve None (o el default que indiques).
.items()	Devuelve todos los pares (key, value). El método más útil para iterar.
.keys()	Devuelve solo las llaves del diccionario.
.values()	Devuelve solo los valores del diccionario.
.pop(key)	Elimina la key y devuelve su valor. Si no existe, genera error (a menos que indiques un default).
.update(d2)	Combina otro diccionario. Si hay keys iguales, sobrescribe los valores con los del segundo.

Diccionario vs Lista — ¿Cuándo usar cada uno?

Característica	Lista []	Diccionario { }
Acceso	Por índice numérico (0, 1, 2...)	Por llave con nombre (key)
Búsqueda	Lenta — recorre todos	Rápida — va directo a la key
Duplicados	Permite valores repetidos	Keys únicas, values repetibles
Uso ideal	Secuencias de datos similares	Datos con etiquetas o nombre
Ejemplo	notas = [80, 90, 70]	persona = {"edad": 25}

 **Regla rápida:** ¿Tus datos tienen etiquetas o nombres? → Usa diccionario. ¿Son una secuencia sin nombre? → Usa lista.

PARTE 1: EJERCICIOS GUIADOS

Sigue cada paso como si estuvieras con un tutor a tu lado

Tema 1: Crear y Acceder a un Diccionario

Un diccionario agrupa datos relacionados bajo un mismo nombre. En lugar de tener tres variables separadas (nombre, edad, ciudad), las encapsulas en un solo objeto con etiquetas descriptivas. Esto hace el código más organizado y fácil de leer.

Ejercicio 1.1 — Mi primer diccionario: datos de un estudiante

Enunciado

Crema un diccionario llamado estudiante con los siguientes datos: nombre, edad, ciudad y promedio. Luego accede a cada valor de dos formas diferentes e imprime todos los pares key-value.

Guía paso a paso:

Paso 1: Crea el diccionario con llaves { } y pares key: value.

```
estudiante = {  
    "nombre": "Carlos",  
    "edad": 20,  
    "ciudad": "Cúcuta",  
    "promedio": 4.2  
}
```

Las llaves { } indican que es un diccionario. Cada par se escribe como "key": value y se separa con coma. La key siempre va entre comillas si es texto. El value puede ser cualquier tipo: texto, número, booleano, lista, etc.

Paso 2: Accede a los valores con corchetes [] y con .get().

```
# Forma 1: con corchetes (directa)  
print(estudiante["nombre"]) # → Carlos  
print(estudiante["promedio"]) # → 4.2  
  
# Forma 2: con .get() (segura)  
print(estudiante.get("ciudad")) # → Cúcuta  
print(estudiante.get("correo", "N/A")) # → N/A (no existe la key)
```

[] vs .get() — diferencia importante:

`estudiante["telefono"]` → lanza `KeyError` si la key no existe.

`estudiante.get("telefono")` → devuelve `None` sin error.

`estudiante.get("telefono", "Sin teléfono")` → devuelve el mensaje default.

Paso 3: Imprime todos los datos con .items().

```
for clave, valor in estudiante.items():  
    print(f"{clave}: {valor}")  
  
# Salida:  
# nombre: Carlos  
# edad: 20
```

```
# ciudad: Cúcuta
# promedio: 4.2
```

.items() desempaqueta cada par en dos variables (clave y valor). Es la forma más común y limpia de recorrer un diccionario.

Mini Reto 1.1

- Agrega una nueva key 'semestre' con el valor 3 sin crear el diccionario de nuevo.
- Cambia el promedio a 4.5 usando la misma sintaxis.
- Imprime el diccionario completo y verifica los cambios.

Pista: Para agregar o modificar → estudiante["nueva_key"] = valor

Solución sugerida — Ejercicio 1.1

```
estudiante = {
    "nombre": "Carlos",
    "edad": 20,
    "ciudad": "Cúcuta",
    "promedio": 4.2
}

# Acceso con []
print(estudiante["nombre"])    # Carlos

# Acceso seguro con .get()
print(estudiante.get("correo", "Sin correo"))    # Sin correo

# Mini reto: agregar y modificar
estudiante["semestre"] = 3
estudiante["promedio"] = 4.5

# Imprimir todo
for k, v in estudiante.items():
    print(f"{k}: {v}")
```

Explicación:

- Si la key ya existe, asignarle un nuevo valor la sobrescribe. Si no existe, la crea.
- No hay un método especial para agregar — se hace igual que modificar.
- .items() es el método favorito para iterar porque te da ambos: key y value.

Ejercicio 1.2 — Tienda de productos

Enunciado

Crea un diccionario de productos con sus precios. Pide al usuario un producto y una cantidad. Si el producto existe, muestra el total a pagar. Si no existe, avisa al usuario. Al final muestra todos los productos disponibles.

Guía paso a paso:

Paso 1: Crea el diccionario de productos.

```
productos = {
    "Menta": 300,
    "Chocorrano": 1000,
    "Esfero": 2700,
    "Chocolatina": 2500
}
```

Las keys son los nombres de los productos (strings). Los values son los precios (enteros). Los diccionarios son perfectos para este tipo de catálogos.

Paso 2: Pide los datos al usuario.

```
producto = input('¿Qué producto deseas? ').title()
cantidad = int(input('¿Cuántos? '))
```

.title() convierte la primera letra en mayúscula para que coincida con las keys del diccionario. Si el usuario escribe 'menta', .title() lo convierte a 'Menta'.

Paso 3: Verifica si el producto existe y calcula el total.

```
if producto in productos:
    total = productos[producto] * cantidad
    print(f'Total a pagar: ${total}')
else:
    print('Producto no disponible')
```

El operador in verifica si una key existe en el diccionario. Es la forma correcta de evitar un KeyError antes de acceder.

Paso 4: Muestra todos los productos disponibles.

```
print('\nProductos disponibles:')
for nombre, precio in productos.items():
    print(f' {nombre}: ${precio}')
```

Mini Reto 1.2

- Agrega dos productos nuevos al diccionario después de crearlo.
- Si el usuario pide más de 5 unidades, aplica un descuento del 10% al total.

Pista: $\text{total} = \text{total} * 0.9$

Solución sugerida — Ejercicio 1.2

```
productos = {
    "Menta": 300, "Chocorrano": 1000,
    "Esfero": 2700, "Chocolatina": 2500
}
```

```
# Mini reto: agregar productos
productos["Chicle"] = 200
productos["Jugo"] = 3500

producto = input('¿Qué producto? ').title()
cantidad = int(input('¿Cuántos? '))

if producto in productos:
    total = productos[producto] * cantidad
    if cantidad > 5:
        total = total * 0.9
        print('Descuento del 10% aplicado')
    print(f'Total: ${total}')
else:
    print('Producto no disponible')

print('\nProductos disponibles:')
for nombre, precio in productos.items():
    print(f' {nombre}: ${precio}')
```

Explicación:

- `in` verifica existencia de la key sin acceder al valor — siempre úsalo antes de `[]`.
- Puedes agregar claves en cualquier momento: `diccionario[nueva_key] = valor`.
- `.title()` es una forma elegante de normalizar la entrada del usuario.

Tema 2: Iterar Diccionarios

Recorrer un diccionario es una de las operaciones más frecuentes en Python. Tienes tres formas según lo que necesites: solo las keys, solo los valores, o ambos juntos.

Ejercicio 2.1 — Inventario de tienda con totales

Enunciado

Tienes un diccionario con el inventario de una tienda (producto: cantidad). Recorre el diccionario con `.items()` para mostrar cada producto con sus unidades, y al final muestra el total de unidades en bodega.

Guía paso a paso:

Paso 1: Crea el inventario.

```
inventario = {
    "Mentas":      10,
    "Chocorramos": 12,
    "Esferos":     2,
    "Chocolatinas": 4
}
```

Paso 2: Recorre con `.items()` y acumula el total.

```
total = 0

for producto, cantidad in inventario.items():
    print(f"{producto}: {cantidad} unidades")
    total += cantidad

print(f"\nTotal en bodega: {total} unidades")

# Salida:
# Mentas: 10 unidades
# Chocorramos: 12 unidades
# Esferos: 2 unidades
# Chocolatinas: 4 unidades
# Total en bodega: 28 unidades
```

`.items()` desempaqueta cada par en dos variables (producto, cantidad). El acumulador `total += cantidad` suma las cantidades en cada vuelta del `for`.

Paso 3: Muestra solo las keys y solo los valores por separado.

```
# Solo keys (nombres de productos)
print('Productos disponibles:')
for producto in inventario.keys():
    print(f'  - {producto}')

# Solo values (cantidades)
print('\nCantidades:')
for cantidad in inventario.values():
    print(f'  {cantidad} unidades')
```

Mini Reto 2.1

- Muestra solo los productos con menos de 5 unidades (stock bajo).

- Cuenta cuántos productos tienen stock suficiente (≥ 5) y cuántos no.

Pista: agrega un if dentro del for para filtrar.

Solución sugerida — Ejercicio 2.1

```
inventario = {
    "Mentas": 10, "Chocorramos": 12,
    "Esferos": 2, "Chocolatinas": 4
}

total = 0
con_stock = 0
sin_stock = 0

for producto, cantidad in inventario.items():
    print(f"{producto}: {cantidad} unidades")
    total += cantidad
    if cantidad >= 5:
        con_stock += 1
    else:
        sin_stock += 1
    print(f" ⚠ Stock bajo: {producto}")

print(f"\nTotal: {total} unidades")
print(f"Con stock: {con_stock} | Stock bajo: {sin_stock}")
```

Explicación:

- Combinar for con if dentro del diccionario es una de las operaciones más comunes.
- `.keys()` es técnicamente redundante — iterar el diccionario sin método ya recorre las keys.
- `.values()` es útil cuando solo necesitas los datos, sin importar las etiquetas.

Ejercicio 2.2 — Registro de notas por materia

Enunciado

Crea un diccionario donde cada key es una materia y el value es la nota. Calcula el promedio, la materia con la nota más alta y la materia con la nota más baja — todo con un for manual sin usar `max()` ni `min()` directamente sobre el diccionario.

Guía paso a paso:

Paso 1: Crea el diccionario de notas.

```
notas = {
    "Matemáticas": 4.5,
    "Python": 3.8,
    "Bases de Datos": 4.0,
    "Inglés": 3.2,
    "Lógica": 4.8
}
```

Paso 2: Calcula el promedio con acumuladores.

```

suma = 0
total = 0

for materia, nota in notas.items():
    suma += nota
    total += 1

promedio = suma / total
print(f"Promedio general: {promedio:.2f}")

```

Paso 3: Encuentra la nota más alta y más baja manualmente.

```

mejor_materia = ''
mejor_nota = 0
peor_materia = ''
peor_nota = 6 # valor inicial mayor que cualquier nota posible

for materia, nota in notas.items():
    if nota > mejor_nota:
        mejor_nota = nota
        mejor_materia = materia
    if nota < peor_nota:
        peor_nota = nota
        peor_materia = materia

print(f"Mejor materia: {mejor_materia} ({mejor_nota})")
print(f"Materia difícil: {peor_materia} ({peor_nota})")

```

Mini Reto 2.2

- Muestra solo las materias donde la nota es menor a 4.0 (para reforzar).
- Cuenta cuántas materias aprobó el estudiante (nota ≥ 3.0).

Solución sugerida — Ejercicio 2.2

```

notas = {"Matemáticas":4.5, "Python":3.8, "Bases de Datos":4.0, "Inglés":3.2,
"Lógica":4.8}

suma = total = aprobadas = 0
mejor_materia = peor_materia = ''
mejor_nota = 0
peor_nota = 6

for materia, nota in notas.items():
    suma += nota
    total += 1
    if nota >= 3.0: aprobadas += 1
    if nota < 4.0:
        print(f"Reforzar: {materia} ({nota})")
    if nota > mejor_nota:
        mejor_nota = nota; mejor_materia = materia
    if nota < peor_nota:
        peor_nota = nota; peor_materia = materia

print(f"Promedio: {suma/total:.2f}")
print(f"Aprobadas: {aprobadas}/{total}")
print(f"Mejor: {mejor_materia} | Difícil: {peor_materia}")

```

Explicación:

- Inicializar la variable del mínimo con un valor mayor que cualquier dato posible es el patrón clásico para encontrar mínimos manualmente.
- Se puede acumular suma, total y aprobadas en el mismo for — no hay que hacer tres ciclos.

Tema 3: Métodos del Diccionario

Los diccionarios tienen métodos específicos para consultar, modificar y eliminar datos. Conocerlos bien te ahorra mucho código.

Ejercicio 3.1 — Agenda de contactos con métodos

Enunciado

Crema una agenda de contactos donde puedas agregar, eliminar y fusionar contactos. Usa los métodos `.pop()`, `.update()` y `.get()` para demostrar su funcionamiento.

Guía paso a paso:

Paso 1: Crea la agenda inicial.

```
agenda = {
    "Ana": "301-555-1111",
    "Luis": "315-555-2222",
    "María": "320-555-3333"
}
```

Paso 2: Usa `.get()` para buscar un contacto de forma segura.

```
nombre = input('¿A quién buscas? ').title()

telefono = agenda.get(nombre, "Contacto no encontrado")
print(f"{nombre}: {telefono}")
```

`.get()` con un segundo argumento devuelve ese default si la key no existe. Evitas el error y no necesitas un `if` previo.

Paso 3: Elimina un contacto con `.pop()` y guarda el número eliminado.

```
eliminado = agenda.pop("Luis")
print(f"Luis eliminado. Su número era: {eliminado}")
print(agenda)
```

`.pop(key)` elimina la key del diccionario y devuelve su valor. Útil cuando necesitas saber qué se eliminó.

Paso 4: Fusiona con otra agenda usando `.update()`.

```
nuevos_contactos = {
    "Carlos": "300-555-4444",
    "Ana": "311-555-9999" # Ana ya existe → se sobrescribe
}

agenda.update(nuevos_contactos)
print(agenda)
# Ana ahora tiene el número nuevo: 311-555-9999
```

`.update()` agrega todos los pares del segundo diccionario. Si una key ya existe, sobrescribe el valor.

Mini Reto 3.1

- Usa `.keys()` para mostrar todos los nombres de la agenda al usuario antes de buscar.
- Agrega una función que muestre cuántos contactos hay con `len(agenda)`.

Pista: `len(diccionario)` cuenta los pares key-value.

Solución sugerida — Ejercicio 3.1

```
agenda = {
    "Ana": "301-555-1111",
    "Luis": "315-555-2222",
    "María": "320-555-3333"
}

# Mostrar contactos disponibles
print(f"Tienes {len(agenda)} contactos:")
for nombre in agenda.keys():
    print(f"    - {nombre}")

# Buscar
nombre = input('¿A quién buscas? ').title()
telefono = agenda.get(nombre, "No encontrado")
print(f"{nombre}: {telefono}")

# Eliminar
eliminado = agenda.pop("Luis")
print(f"Eliminado: {eliminado}")

# Fusionar
agenda.update({"Carlos": "300-555-4444", "Ana": "311-555-9999"})
print(f"Agenda actualizada: {len(agenda)} contactos")
```

Explicación:

- `.pop()` devuelve el valor — `.pop(key, default)` no lanza error si la key no existe.
- `.update()` modifica el diccionario original. No crea uno nuevo.
- `len(diccionario)` cuenta los pares key-value, igual que `len(lista)` cuenta elementos.

Tema 4: Diccionarios Anidados

Un diccionario puede contener otros diccionarios como valores. Esto permite modelar datos más complejos, como productos con múltiples propiedades, estudiantes con varias notas, o usuarios con información detallada.

Ejercicio 4.1 — Inventario anidado de tienda

Enunciado

Crea un diccionario anidado donde cada producto tiene costo, precio y cantidad disponible. Accede a propiedades específicas, calcula la ganancia de cada producto si se vende todo, y muestra el producto más rentable.

Guía paso a paso:

Paso 1: Crea el diccionario anidado.

```
productos = {
    "Menta": {
        "costo": 40,
        "precio": 300,
        "cantidad": 10
    },
    "Chocorrano": {
        "costo": 700,
        "precio": 1000,
        "cantidad": 12
    },
    "Esfero": {
        "costo": 1500,
        "precio": 2700,
        "cantidad": 5
    }
}
```

Cada producto es un diccionario dentro del diccionario principal. La key del producto lleva a su propio diccionario de propiedades.

Paso 2: Accede a propiedades con doble [][].

```
# Precio de la Menta
print(productos["Menta"]["precio"])      # → 300

# Cantidad de Chocorrano
print(productos["Chocorrano"]["cantidad"]) # → 12

# Costo del Esfero
print(productos["Esfero"]["costo"])      # → 1500
```

El primer [] selecciona el producto (diccionario interno). El segundo [] selecciona la propiedad dentro de ese diccionario.

Paso 3: Calcula la ganancia de cada producto.

```
mejor_producto = ''
mejor_ganancia = 0

for nombre, datos in productos.items():
    ganancia = (datos['precio'] - datos['costo']) * datos['cantidad']
```

```

print(f"{nombre}: ganancia = ${ganancia}")
if ganancia > mejor_ganancia:
    mejor_ganancia = ganancia
    mejor_producto = nombre

print(f"\nProducto más rentable: {mejor_producto} (${mejor_ganancia})")

```

En cada iteración, nombre es el nombre del producto y datos es su diccionario interno. Accedes a datos['precio'] igual que accederías a cualquier diccionario.

Mini Reto 4.1

- Simula una venta: pide un producto y una cantidad al usuario. Si hay suficiente stock, descuenta la cantidad de 'cantidad' y muestra el total cobrado.
- Si no hay suficiente stock, avisa cuántas unidades quedan disponibles.

Pista: productos[producto]['cantidad'] -= cantidad_vendida

Solución sugerida — Ejercicio 4.1

```

productos = {
    "Menta": {"costo":40, "precio":300, "cantidad":10},
    "Chocorrano": {"costo":700, "precio":1000, "cantidad":12},
    "Esfero": {"costo":1500, "precio":2700, "cantidad":5}
}

# Calcular ganancias
mejor_producto = ""
mejor_ganancia = 0

for nombre, datos in productos.items():
    g = (datos["precio"] - datos["costo"]) * datos["cantidad"]
    print(f"{nombre}: ${g}")
    if g > mejor_ganancia:
        mejor_ganancia = g
        mejor_producto = nombre

print(f"Más rentable: {mejor_producto}")

# Mini reto: simular venta
producto = input("¿Qué producto? ").title()
cantidad = int(input("¿Cuántos? "))

if producto in productos:
    stock = productos[producto]["cantidad"]
    if cantidad <= stock:
        productos[producto]["cantidad"] -= cantidad
        total = productos[producto]["precio"] * cantidad
        print(f"Venta exitosa. Total: ${total}")
    else:
        print(f"Stock insuficiente. Solo hay {stock} unidades.")
else:
    print("Producto no encontrado.")

```

Explicación:

- datos es el diccionario interno — lo tratas exactamente igual que cualquier diccionario.
- Modificar productos[producto]['cantidad'] -= n modifica el dato directamente dentro del diccionario anidado.
- Este patrón (diccionario de diccionarios) es muy común para representar registros con múltiples campos.

PARTE 2: EJERCICIOS INDEPENDIENTES

Crear y Acceder: Practica lo aprendido sin guía — Solo enunciado y datos de prueba

Ejercicio P1 — Ficha de película

Enunciado

Crea un diccionario película con: título, director, año y género. Muestra el título con [] y el director con .get(). Luego agrega una key 'calificacion' con un número del 1 al 10 e imprime todo el diccionario con .items().

Ejemplo de uso	Salida esperada
pelicula['titulo']	"Interstellar"
pelicula.get('director')	"Nolan"

Pista: Recuerda: para agregar → pelicula['nueva_key'] = valor

Ejercicio P2 — Conversor de monedas

Enunciado

Crea un diccionario con tasas de cambio: dolar, euro y libra (en pesos colombianos). Pide al usuario qué moneda quiere convertir y cuánto dinero tiene. Calcula y muestra el equivalente en pesos. Si la moneda no existe, avisa con .get().

Ejemplo de uso	Salida esperada
Moneda: dolar Monto: 100	Equivalente: \$412.000

Pista: Usa .get(moneda, None) para verificar antes de calcular.

Ejercicio P3 — Contador de palabras

Enunciado

Pide una frase al usuario. Separa las palabras con .split() y crea un diccionario donde cada key es una palabra y el value es cuántas veces aparece. Al final muestra las palabras y sus conteos.

Ejemplo de uso	Salida esperada
Frase: "hola mundo hola"	{ 'hola': 2, 'mundo': 1 }

Pista: Para cada palabra: if palabra in conteo → conteo[palabra] += 1, else → conteo[palabra] = 1

Iterar Diccionarios

Ejercicio P4 — Reporte de ventas

Enunciado

Tienes un diccionario con ventas por mes (enero: 1500000, febrero: 2300000, marzo: 980000, abril: 3100000). Recórrelo con `.items()` y muestra cada mes con su venta. Al final muestra el mes con mayor venta, el mes con menor venta y el total acumulado — todo con un `for` manual.

Ejemplo de uso	Salida esperada
enero: \$1.500.000	...
Mejor mes: abril Total: \$7.880.000	

Pista: Inicializa el mínimo con un número muy grande (ej: 999999999).

Ejercicio P5 — Clasificar estudiantes

Enunciado

Tienes un diccionario con nombres y notas. Recórrelo e imprime tres listas separadas: Excelente (≥ 4.5), Aprobado (≥ 3.0 y < 4.5), y Reprobado (< 3.0). Muestra cuántos estudiantes hay en cada categoría.

Ejemplo de uso	Salida esperada
{'Ana':4.8, 'Luis':3.2, 'María':2.5}	Excelente: AnaAprobado: LuisReprobado: María

Pista: Usa `if/elif/else` dentro del `for` y un acumulador por categoría.

Ejercicio P6 — Diccionario desde listas

Enunciado

Tienes dos listas: una de materias y una de notas. Crea un diccionario que combine ambas usando un `for` con `range()`. Luego recorre el diccionario e imprime solo las materias con nota ≥ 4.0 .

Ejemplo de uso	Salida esperada
materias=['Mate','Python','Inglés']notas=[4.5, 3.8, 4.2]	{'Mate':4.5, 'Python':3.8, 'Inglés':4.2}Mate: 4.5Inglés: 4.2

Pista: Para construir: `registro[materias[i]] = notas[i]` dentro de `for i in range(len(materias))`.

Métodos del Diccionario

Ejercicio P7 — Menú de restaurante dinámico

Enunciado

Crea un menú de restaurante como diccionario. Implementa tres opciones: (1) mostrar todos los platos con precios, (2) eliminar un plato con `.pop()`, (3) actualizar precios con `.update()`. El programa debe repetirse hasta que el usuario elija salir.

Ejemplo de uso	Salida esperada
Opción 1	Plato A: \$15000Plato B: \$22000...
Opción 2: eliminar 'Plato A'	Plato A eliminado. Precio era: \$15000

Pista: Usa `while True` con `if/elif` para el menú. `.pop(plato, 'No existe')` evita errores.

Ejercicio P8 — Fusionar dos inventarios

Enunciado

Tienes dos inventarios de dos bodegas (diccionarios). Fusi6nalos con `.update()` en un inventario combinado. Si un producto aparece en ambas bodegas, el inventario final debe tener la SUMA de cantidades, no sobrescribir. Muestra el inventario final.

Ejemplo de uso	Salida esperada
bodega1={'Menta':10,'Esfero':5}bodega2={'Menta':8,'Cuaderno':20}	Inventario:Menta: 18Esfero: 5Cuaderno: 20

Pista: No uses `.update()` directamente (sobrescribe). Recorre `bodega2` con `for` y suma manualmente si la `key` ya existe.

Diccionarios Anidados

Ejercicio P9 — Sistema de estudiantes

Enunciado

Crea un diccionario donde cada key es el nombre de un estudiante y su value es otro diccionario con: nota1, nota2, nota3 y ciudad. Calcula el promedio de cada estudiante. Muestra quién aprobó (promedio ≥ 3.0) y quién reprobó. Al final, muestra el estudiante con el mejor promedio general.

Ejemplo de uso	Salida esperada
<code>{'Ana':{'nota1':4.5,'nota2':3.8,'nota3':4.0,'ciudad':'Cúcuta'},...}</code>	Ana: promedio 4.1 — Aprobado...

Pista: Para el promedio: $(\text{datos}[\text{'nota1'}] + \text{datos}[\text{'nota2'}] + \text{datos}[\text{'nota3'}]) / 3$

Ejercicio P10 — Catálogo de celulares

Enunciado

Crea un diccionario anidado con al menos 3 celulares. Cada celular tiene: marca, precio, ram y almacenamiento. El programa debe permitir: (1) ver todos los celulares con sus especificaciones, (2) buscar un celular por nombre y mostrar sus detalles, (3) filtrar y mostrar solo los celulares con precio $\leq$ al presupuesto que el usuario indique.

Ejemplo de uso	Salida esperada
iPhone 15: \$3.200.000 6GB RAM 128GB	...
Presupuesto: \$2.000.000	Samsung A54: \$1.800.000...

Pista: Para recorrer especificaciones: `for spec, val in celulares[nombre].items()`

Ejercicio Combinado — Sistema de Facturación

Enunciado

Construye un sistema de facturación completo usando solo diccionarios. El programa permite registrar ventas, calcular totales y generar un reporte final.

Estructura de datos:

```
catalogo = {
    "Menta": {"precio": 300, "stock": 50},
    "Chocorrano": {"precio": 1000, "stock": 30},
    "Esfero": {"precio": 2700, "stock": 20},
    "Chocolatina": {"precio": 2500, "stock": 15}
}

factura = {} # aquí se registra lo que compra el cliente
```

El programa debe hacer:

1. Mostrar el catálogo disponible con precios y stock.
2. Pedir al usuario qué producto quiere y cuántas unidades.
3. Verificar si hay suficiente stock. Si sí, agregar a la factura y descontar del stock.
4. Repetir hasta que el usuario escriba 'fin'.
5. Al terminar, mostrar la factura: cada producto, cantidad, subtotal y el TOTAL GENERAL.

Pista: La factura también es un diccionario: `factura[producto] = {'cantidad': x, 'subtotal': y}`. Si el usuario agrega el mismo producto dos veces, suma las cantidades en lugar de sobrescribir.

¡Felicidades por completar los ejercicios!

"La llave correcta abre cualquier puerta — y ahora conoces todas las llaves"

Los diccionarios son la estructura más usada en Python profesional.

Úsalos cuando tus datos tengan nombre. Combínalos con listas y funciones para construir programas poderosos.